

Roteiro — Aula 4: Servidor, Docker, Kamal e deploy

Deck: `slides/build/aula-04-servidor-docker-kamal-e-deploy.pptx` (71 slides, sendo 8 de prática)
Apostila da turma: `04-deploy.md` · **Checkpoint:** `aula-04` **Antes de tudo:** `guia-do-instrutor.md` — conduzir a sala, não o conteúdo **Apêndice:** `apendice-azure.md` — o mesmo percurso na nuvem, para casa

Esta aula era a mais arriscada das quatro, porque o gargalo não era digitar: era clicar em portal, e portal falha. **Agora o servidor é uma VM no notebook de cada um.** O risco caiu muito, e o que sobrou dele está concentrado num lugar só — quem usa Windows — com duas correções conhecidas e um script que descobre o caso uma semana antes.

O que preparar com antecedência

#	O que	Quando
1	Cobrar de todo mundo a saída do <code>checar-servidor.sh</code> , com print	uma semana antes
2	Resolver, um a um, os que voltarem com erro (é sempre Windows, e são dois casos)	uma semana antes
3	Uma conta Resend (ou outro SMTP) com domínio verificado, e uma API key para a turma	véspera
4	Fazer o percurso inteiro sozinho, do <code>multipass launch</code> ao <code>curl --cacert</code> , com uma VM nova	véspera

O item 1 é o que tira o risco da aula

O Multipass roda no Windows; o Rails deles roda no WSL2. São duas máquinas virtuais diferentes, e **por padrão uma não enxerga a outra** — o `ssh` do WSL2 não alcança o IP da VM, e sem isso nada nesta aula funciona.

A boa notícia é que isso é **detectável e corrigível uma semana antes, sem você**. Peça no grupo:

```
multipass launch 24.04 --name teste
multipass info teste                # anote o IPv4
cd ~/capacitacao-gabarito
./scripts/checar-servidor.sh <IP>  # print disto, no grupo
multipass delete teste && multipass purge
```

O script testa a porta, testa o SSH com a chave, e quando falha imprime a correção do caso dele. **Você só precisa olhar os prints e cobrar quem não mandou.**

As duas correções, para você reconhecer de longe:

Caso	Correção	Onde
Windows 11 22H2+	<code>networkingMode=mirrored</code> no <code>.wslconfig</code> , depois <code>wsl --shutdown</code>	um arquivo
Windows 10, ou o mirrored não pegou	<code>netsh interface portproxy</code> para 2222/443/80, mais a regra de firewall	PowerShell admin

No segundo caso o `SERVER_IP` deles vira o IP do **Windows** (`ip route show default`), e o `.env` ganha `SSH_PORT=2222`. O `deploy.yml` **já lê essa variável** — o Kamal funciona sem mais nenhuma mudança, e os comandos `ssh` da apostila só precisam de `-p 2222`.

Os dois estão escritos em `00-preparacao.md` e em `troubleshooting.md`. Mande o link, não explique você mesmo doze vezes.

Só se as duas falharem (raro, e você vai saber com uma semana de antecedência): dupla. Duas pessoas numa máquina que funciona aprendem; uma pessoa travada na rede não aprende nada.

Antes de começar

- [] Sua VM de teste **destruída** (`multipass delete servidor && multipass purge`), para você fazer o percurso do zero junto com eles.
- [] Dois terminais grandes, lado a lado: um é "o seu notebook", o outro é "dentro da VM". Diga isso em voz alta e mantenha a divisão a aula inteira — metade da confusão do dia é aluno rodando na máquina errada.
- [] Um navegador aberto, para o momento do aviso de certificado.
- [] `docs/troubleshooting.md` aberto numa aba.
- [] Portal da Azure logado numa aba anônima, para a demo do fim (não vaze recursos do SEEM na projeção).
- [] O PAT do ghcr.io: avise no grupo, na véspera, para já virem com ele criado.

Cronograma

As oito práticas são o esqueleto do dia. Estão em negrito, e nesta aula elas são quase tudo: o conteúdo existe para explicar o que a mão está fazendo.

Relógio	Slides	Bloco	Min
00:00	1–3	Abertura, e de onde paramos	6
00:06	4–10	O servidor, as opções, por que não Kubernetes	16
00:22	11–17	A máquina: chaves, <code>cloud-init</code> , <code>multipass</code>	14
00:36	18	Prática 1 — subir a VM e entrar nela	20
00:56	19–27	Ubuntu: firewall, Docker, root, swap, SSH	24
01:20	28	Prática 2 — firewall, Docker, swap, SSH	30
01:50	—	Intervalo	10
02:00	29–34	Docker: imagem, Dockerfile, registry, arquitetura	16
02:16	35	Prática 3 — arquivos de deploy e o token	15
02:31	36–44	Kamal: <code>deploy.yml</code> , accessory, segredos	22
02:53	45	Prática 4 — o <code>.env</code>	15
03:08	46–53	TLS, certificado, CA, proxy reverso	22
03:30	54	Prática 5 — certificado e <code>/etc/hosts</code>	15
03:45	55–59	CI, o primeiro deploy, operar, backup	14
03:59	60–61	Práticas 6 e 7 — o deploy, e o certificado com os olhos	35
04:34	62–65	A nuvem de verdade: demo	15
04:49	66	Prática 8 — o sistema inteiro, e o backup	20
05:09	67–71	O que vocês fizeram, e agora, fim	8

Dá 5h17 com tudo, e é honesto dizer: esta aula tem duas horas de mão na massa que não dá para comprimir — o `apt`, o build da imagem e o download do Ubuntu levam o tempo que levam.

Corte **nesta ordem**:

#	O que cortar	Ganho
1	Prática 8 vira dever de casa (fluxo da Aula 3 em produção, e o backup)	–20
2	Slide 20 (<code>LINUX BÁSICO</code> , # <code>CORTÁVEL</code>) se a turma já usa terminal	–4
3	A demo de nuvem (62–65) encolhe para 6 min: só o slide 63 e o 64	–9
4	Prática 2: dê o bloco do Docker num script pronto, para colarem	–12
5	Slides 31–32 (Dockerfile e as duas decisões) — ficam na apostila	–8
6	Prática 5: você gera um certificado e distribui; eles só fazem o <code>/etc/hosts</code>	–8
7	Slides 43–44 (credentials × ENV, três camadas) — ficam na apostila	–8

Cortando de 1 a 3, fecha em **4h44**. Cortando os sete, **3h48**.

Nunca corte as Práticas 6 e 7. Ver a API própria no ar e entender o aviso de certificado são os dois momentos que a turma leva embora.

Onde o tempo ainda pode escapar

Risco	Sinal	O que fazer
Windows sem rede para a VM	<code>ssh</code> dá timeout no aluno do Windows	Rode o <code>checar-servidor.sh</code> nele e aplique a correção que o script imprimir. Se não sair em 5 min, dupla, e siga — não debugue rede na frente da turma.
Download da imagem do Ubuntu	<code>multipass launch</code> parado em "Retrieving image"	Wi-Fi do local. Peça para começarem o launch no slide 11 , antes da teoria de chaves.
Build da imagem Docker	<code>kamal setup</code> demorando	Normal: ~5 min no primeiro. Use o tempo para o slide de backup e para perguntas.

Truque que devolve 10 minutos: mande rodar o `multipass launch` no slide 11 e deixe baixando enquanto você dá a teoria de chaves (13–14). Quando a teoria acabar, a VM está pronta.

Plano B, decidido de véspera

B1 — Cortar o TLS. Se às 03:08 a maioria ainda não deployou, tire `proxy.ssl` do `deploy.yml`, suba em HTTP puro no IP da VM, e faça o bloco de TLS todo projetado, no seu. **Devolve ~20 min.** Eles refazem em casa com a apostila.

B2 — Virar demo a partir do intervalo. Se às 01:50 não houver pelo menos metade da turma com VM respondendo ao SSH, pare de esperar. Projete o percurso do começo ao fim. **Melhor todo mundo ver funcionando do que metade travar em rede.**

Bloco a bloco

00:06 — O servidor (4–10)

A frase que abre e fecha a aula está no slide 4, e vale repetir três vezes ao longo do dia:

Muda o IP e a conta no fim do mês. Não muda o que você faz.

O slide 6 responde a pergunta que alguém vai fazer antes de você chegar nela ("por que não roda no meu Ubuntu mesmo?"): porque servidor tem que ser descartável, e porque container não tem usuário, firewall, `systemd` nem `sshd` — que é metade da aula.

O slide 10 é o que dá credibilidade: admita o custo. Uma VM é ponto único de falha, volume não é backup, o Postgres não é gerenciado.

00:22 — A máquina (11–17), e a Prática 1 (18)

Primeira coisa, antes de qualquer slide deste bloco: mande todo mundo rodar o `multipass launch` (slide 17, projete-o já). Deixe baixando. Depois volte ao 10 e dê a teoria.

- **13–14** — chave pública e privada. É o conceito que também explica o JWT da Aula 3 e o TLS que vem daqui a pouco. **Não pule**, mesmo com a turma ansiosa para digitar.
- **15** — o `chmod 600`. Diga que o SSH **recusa** usar chave com permissão frouxa, e que é a primeira coisa a conferir quando der `Permission denied (publickey)`.
- **16** — `cloud-init`. Vale uma frase: *"isto não é coisa do Multipass. É o mesmo arquivo que a Azure, a AWS e a DigitalOcean leem quando criam uma máquina."*

Ponto de decisão às 00:56: se menos da metade conectou por SSH, aplique o plano B2.

00:56 — Ubuntu (19–27), e a Prática 2 (28)

- **21–22** — o firewall. O slide 22 é o mais importante do bloco, e o mais desconfortável: **diga que o `ufw` não está protegendo eles de nada hoje**. Turma percebe quando a gente finge que um exercício é uma proteção, e a honestidade aqui compra atenção para o resto.
- **24** — variável de ambiente. É o conceito que sustenta o `env.clear` / `env.secret` do Kamal.
- **26** — swap. Explique o OOM killer. Reconheça que com 2 GiB provavelmente não vai precisar hoje, e que a VPS de US\$5 tem 1 GiB e vai.
- **27** — endurecer o SSH. **Fale isto em voz alta:** *"mantenham a sessão atual aberta e testem em outro terminal."* E logo depois: *"aqui vocês têm o `multipass shell` como rede de segurança. Numa máquina alugada, essa porta não existe — por isso o hábito é agora."*

02:00 — Docker (29–34), e a Prática 3 (35)

Conceitual e rápido. Imagem é receita, container é bolo.

No **32**, as duas decisões: multi-stage (compilador não vai para produção) e usuário não-root (*"é uma linha que muda o tamanho do estrago"*).

O **33** explica por que a imagem sobe para o ghcr.io em vez de ir direto para a VM ao lado: porque é o que aconteceria com uma VPS do outro lado do mundo, e é o que faz o mesmo `deploy.yml` servir nos dois casos. Sem isso, alguém vai perguntar — com razão.

O **34** é operacional: PAT com `write:packages`, e a arquitetura. **Circule pela sala neste slide**: quem tem Mac com chip M precisa de `arm64`, e quem errar isso só descobre no meio do `kamal setup`, com uma mensagem que não diz "arquitetura".

02:31 — Kamal (36–44), e a Prática 4 (45)

Abra o `config/deploy.yml` projetado e percorra junto com os slides.

O slide 40 (`clear` × `secret`) tem o argumento concreto: `JWT_SECRET` no `clear` apareceria em `docker inspect` e no histórico do shell. Se der, mostre um `docker inspect` de verdade.

O **43** (três camadas de segredo) é o slide que amarra tudo. Deixe na tela enquanto explica, e diga a linha que conecta com o apêndice: *"quando isso virar um pipeline, só a primeira camada muda."*

No **44**, pare no `.env`: `git status` **não pode mostrar esse arquivo**. Mande todo mundo rodar, ali, na frente de você.

03:08 — TLS (46–53), e as Práticas 5 a 7 (54, 60–61)

O melhor bloco da aula nova. Antes era teoria sobre Cloudflare; agora eles veem acontecer.

- **46–49** — HTTPS é HTTP num túnel; handshake em três passos; certificado e cadeia de confiança.
- **49** é a virada: *"para uma CA pública assinar, você tem que provar que o domínio é seu. E vocês não têm domínio nenhum apontando para essa VM."* É daí que sai o autoassinado, como consequência, e não como gambiarra.
- **52** — `.test` e `/etc/hosts`. Vale a curiosidade: `.test` é reservado por RFC exatamente para isso, e ninguém consegue registrar.
- **53** — **o slide da aula**. Não passe rápido. Os três `curl` respondem, um a um, o que TLS é e o que CA é:

```
curl https://seunome.test/...      # falha: self signed certificate
curl -k https://seunome.test/...    # funciona, sem conferir nada
curl --cacert tls/capacita-cert.pem ... # funciona, CONFERINDO
```

A frase que fixa: "o `-k` desliga a conferência. O `--cacert` não desliga nada — ele diz em quem confiar. Uma CA é isso, e só isso: alguém em quem o seu sistema já decidiu confiar, de fábrica."

Depois abra no navegador e leia o aviso vermelho junto com eles. **É o mesmo fato, dito para um humano.**

03:45 — CI, deploy e operação (55–59)

```
source .env && bundle exec kamal config
```

Valida sem tocar em nada. **Faça isso antes** — pega variável faltando de graça.

Depois `kamal setup`. Demora ~5 minutos no primeiro build. Use o tempo para o slide de backup (59) e para perguntas.

E então, o momento da aula:

```
curl --cacert tls/capacita-cert.pem https://seunome.test/api/v1/status
```

Diga o que está na frente deles, porque nem todo mundo percebe sozinho: *"o código de vocês está rodando num container, dentro de um Ubuntu que vocês provisionaram, atrás de um proxy TLS, com um Postgres com volume. Isto é produção. Só falta o IP ser público."*

04:34 — A nuvem, demo (62–65)

Quinze minutos, projetado, sem ninguém acompanhando no teclado. Diga isso antes de começar, ou metade da turma vai tentar criar conta na Azure e perder o fim da aula.

Percorra: portal → VM → NSG com a 22 só para o seu IP → DNS no Cloudflare → Origin CA → OIDC. Não detalhe nenhum; o que você quer é que reconheçam as peças e saibam que o passo a passo existe.

O slide 64 (a sequência do deploy automático) é o único que merece um minuto inteiro: abrir a 22, deployar, e **fechar mesmo se falhar**.

E o 65, o OIDC: *"em vez de guardar uma senha que vale meses, o GitHub prova quem é a cada execução."*

Feche apontando o apêndice: [docs/apendice-azure.md](#), e **a Azure for Students dá US\$100 sem cartão**. Quem quiser, faz em casa e me chama.

04:49 — Prática 8 e fecho (66–71)

O slide 68 ([O QUE O SEEM-BACKEND TEM A MAIS](#)) é o convite: *"nada disso é difícil depois do que vocês viram hoje. O código está lá, e agora vocês conseguem ler."*

E então **reserve cinco minutos de verdade para os slides 69 e 70**. Não corra.

O **69** ([O QUE VOCÊS FIZERAM](#)) é a lista do que eles construíram, e existe porque quem fez raramente percebe o tamanho: o esforço esconde a conquista. Leia devagar, olhando para eles.

O **70** ([E AGORA?](#)) deixa o canal aberto. Diga em voz alta que dúvida daqui a três semanas ainda é dúvida — a pergunta que chega depois costuma ser a mais importante da capacitação inteira, porque é a primeira que veio de um problema real deles.

Perguntas que vão aparecer

Pergunta	Resposta curta
"Isso não é 'de verdade', é só no meu computador."	É de verdade: é o mesmo Ubuntu, o mesmo Docker, o mesmo Kamal e o mesmo <code>deploy.yml</code> . O que falta é o IP ser público — e é uma variável no <code>.env</code> .
"Então por que não usar a nuvem direto?"	Porque metade da aula viraria espera de portal e cota, e nada disso é back-end. O apêndice tem o caminho, e vocês já vão saber operar a máquina quando chegarem lá.
"Por que não Heroku/Render/Vercel? É mais fácil."	É mesmo, e para muita coisa é a escolha certa. Aqui o objetivo é vocês entenderem o que essas plataformas fazem por vocês — e o custo delas cresce rápido.
"Por que não rodar só com Docker Compose no meu Ubuntu?"	Compose sobe container. Não resolve máquina descartável, usuário, firewall, <code>sshd</code> , build remoto, zero-downtime, rollback nem segredo.
"E se a VM cair?"	Cai o sistema. É o custo declarado desta arquitetura. Com backup, você sobe outra em 20 min — e aqui, em 2.
"Por que o banco não é gerenciado?"	Custo. Um Postgres gerenciado custa mais que a VM inteira. Para 500 usuários numa semana, não se paga.
"Meu navegador diz que o site não é seguro."	E está certo. O certificado é seu, assinado por você. É o slide 48 — e é a diferença entre criptografia e confiança.
"Posso usar isso num projeto meu?"	Pode, e é a intenção. Troque o nome do serviço, o host e os segredos. O resto é igual.
"Preciso pagar por um domínio?"	Só quando for para a internet. ~R\$15/ano, ou grátis pelo GitHub Student Pack.

Depois da capacitação

Mande no grupo:

1. `multipass stop servidor` quando não estiverem usando — a VM para não come RAM do notebook.
2. `apendice-azure.md`, com o link da Azure for Students, para quem quiser o IP público e o deploy automático.
3. O link do `seem-backend`, com o convite do slide 68.
4. `docs/troubleshooting.md` — é o que eles vão abrir quando algo quebrar sem você por perto.